V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_15

15. Template Method Pattern

Vaskaran Sarcar[1]

(1) Garia, Kolkata, India

This chapter covers the Template Method pattern.

GoF Definition

It defines the skeleton of an algorithm in an operation, deferring some steps to subclasses. The Template Method pattern lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.

Concept

Using this pattern, you begin with the minimum or essential structure of an algorithm. Then you defer some responsibilities to the subclasses. As a result, the derived class can redefine some steps of an algorithm without changing the flow of the algorithm.

Simply put, this design pattern is useful when you implement a multistep algorithm but allow customization through subclasses. Here you keep the outline of the algorithm in a separate method referred to as a **template method** . The container class of this template method is often referred to as the **template class**.

It is important to note that some amount of implementation may not vary across the subclasses. As a result, you may see some default implementations in the template class. Only the specific details are implemented in a subclass. So, using this approach, you write a minimum amount of code in a subclass.

Real-Life Example

When you order a pizza, the chef of the restaurant can use a basic mechanism to prepare the pizza, but they may allow you to select the final mate-

rials. For example, a customer can opt for different toppings such as bacon, onions, extra cheese, or mushrooms. So, just before the delivery of the pizza, the chef can include these choices.

Computer World Example

Suppose you have been hired to design an online engineering degree course. In general, the first semester of the course is the same for all students. For subsequent semesters, you need to add new papers or subjects to the application based on the course opted by a student.

The Template Method pattern makes sense when you want to avoid duplicate code in your application but allow subclasses to change specific details of the parent class workflow to bring varying behavior to the application. (However, you may not want to override the parent class methods entirely to make radical changes in the subclasses. In this way, the pattern differs from simple polymorphism.)

Here is an built-in example for you:

- The `removeAll()` method of `java.util.AbstractSet` can be considered as an example of a Template Method pattern. (You can refer to Q&A 15.9 for additional information).

Implementation

Let's assume that each engineering student needs to pass the courses on Mathematics and soft skills (such as communication skills, people management skills, and so on) in their initial semester to obtain their degrees. Later you will add some special papers to their courses based on their chosen paths (computer science or electronics).

To serve the purpose, a template method `displayCourseStructure()` is defined in an abstract class `BasicEngineering`.

> **Note** You will see that the `displayCourseStructure()` method calls the other methods in sequential order. You can think of them as the steps of the overall algorithm.

It has the following structure. The supporting comments are kept for your easy understanding.

```
abstract class BasicEngineering {
     // The "Template Method"
```

```
      // Making the method final to prevent overriding.
      public final void displayCourseStructure() {
            /*
             * The course needs to be completed
             * in the following sequence:
             * 1.Mathematics
             * 2.Soft skills
             * 3.Subclass-specific paper
             */
            // Common Papers:
            courseOnMathematics();//Step-1
            courseOnSoftSkills(); //Step-2
            // Course-specific Paper:
            courseOnSpecialPaper();//Step-3
      }
      private void courseOnMathematics() {
            System.out.println("1. Mathematics");
      }
      private void courseOnSoftSkills() {
            System.out.println("2. Soft Skills");
      }
      /*
       * The following method will vary.
       * It will be overridden by the
       * derived classes.
       */
      public abstract void courseOnSpecialPaper();
}
```

Notice that subclasses of `BasicEngineering` cannot alter (or override) the flow of the `displayCourseStructure()` method but they can override the `SpecialPaper()` method to include course-specific details and make the final course list different from each other.

The concrete classes `ComputerScience` and `Electronics` are the subclasses of `BasicEngineering` and they take the opportunity to override the `courseOnSpecialPaper()` method . The following code segment shows such a sample from the `ComputerScience` class:

```
// The concrete derived class: ComputerScience
class ComputerScience extends BasicEngineering {
      @Override
      public void courseOnSpecialPaper() {
            System.out.println("3. Object-Oriented Programming");
      }
}
```

Class Diagram

Figure 15-1 shows the important parts of the class diagram.

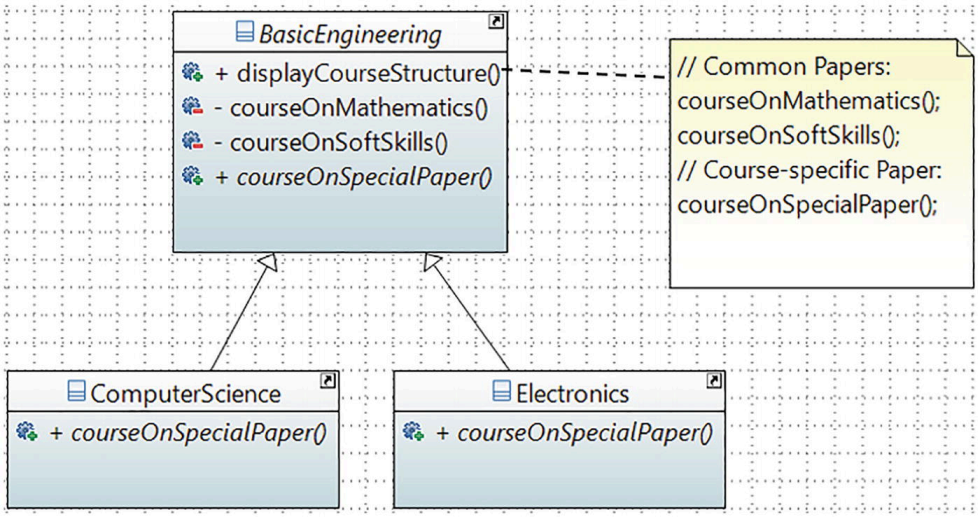

Figure 15-1 Class diagram

Package Explorer View

Figure 15-2 shows the high-level structure of the program.

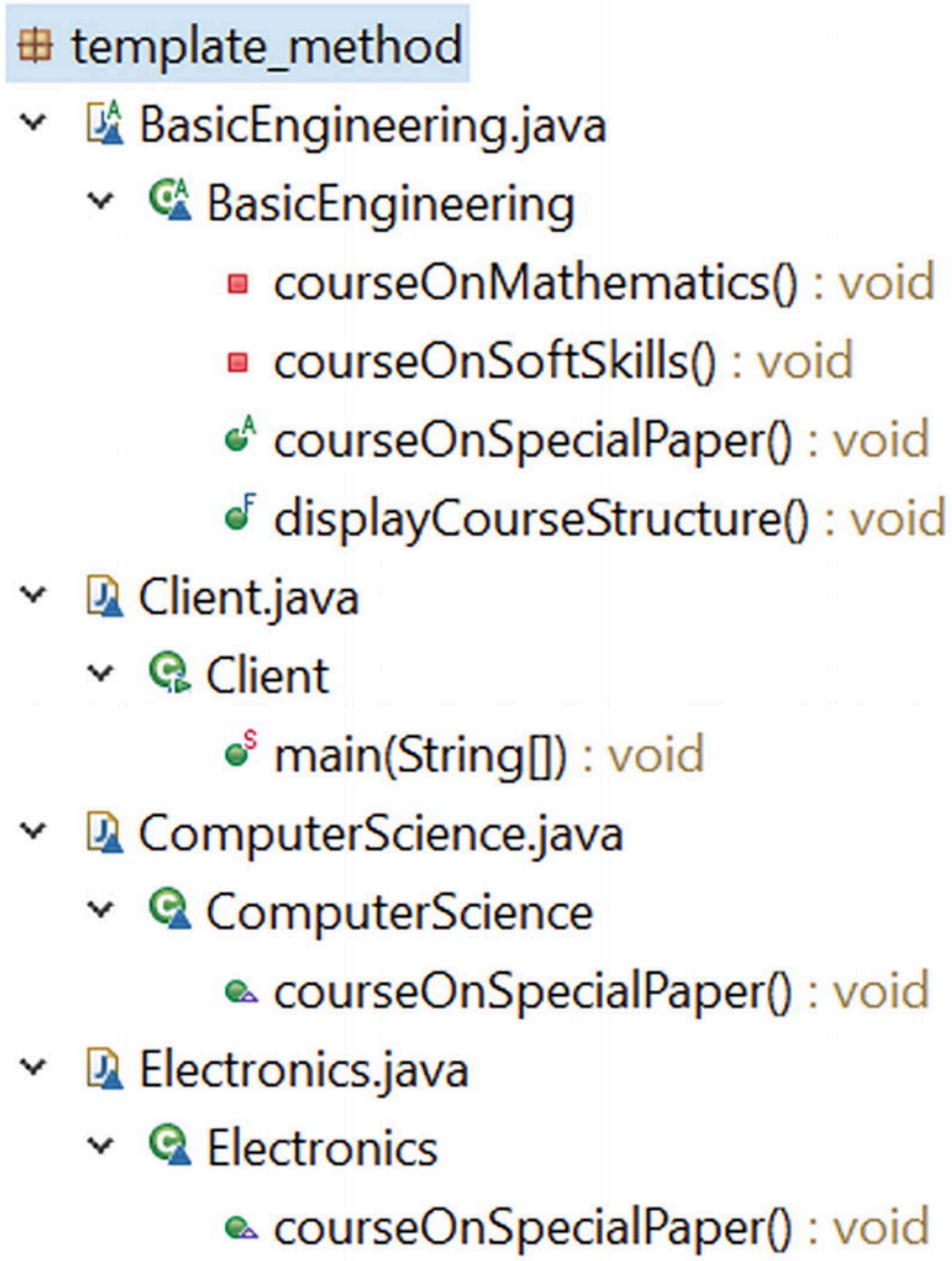

Figure 15-2 Package Explorer view

Demonstration 1

Here's the implementation. All parts of this program are stored in the package `jdp3e.template_method`.

```
// BasicEngineering.java
abstract class BasicEngineering {
     // The "Template Method"
     // Making the method final to prevent overriding.
     public final void displayCourseStructure() {
          /*
           * The course needs to be completed in the
            * following sequence:
            * 1.Mathematics
            * 2.Soft skills
```

```
             * 3.Subclass-specific paper




              */
             // Common Papers:
             courseOnMathematics();
             courseOnSoftSkills();
             // Course-specific Paper:
             courseOnSpecialPaper();
        }
        private void courseOnMathematics() {
             System.out.println("1. Mathematics");
        }
        private void courseOnSoftSkills() {
             System.out.println("2. Soft skills");
        }
        /*
         * The following method will vary.
         * It will be overridden by the
         * derived classes.
         */
        public abstract void courseOnSpecialPaper();
}
// ComputerScience.java
// The concrete derived class: ComputerScience
class ComputerScience extends BasicEngineering {
        @Override
        public void courseOnSpecialPaper() {




             System.out.println("3. Object-Oriented Programming");
        }
}
// Electronics.java
// The concrete derived class: Electronics
class Electronics extends BasicEngineering {
        @Override
        public void courseOnSpecialPaper() {
             System.out.println("3. Digital Logic and Circuit Theory");
        }
}
// Client.java
class Client {
        public static void main(String[] args) {
             System.out.println("***Template Method Pattern Demonstration.***
             BasicEngineering preferrredCourse = new
                                  ComputerScience();
```

```
            System.out.println("Computer Science course structure:");
            preferrredCourse.displayCourseStructure();
            System.out.println();
            preferrredCourse = new Electronics();
            System.out.println("Electronics course structure:");
            preferrredCourse.displayCourseStructure();
        }
    }
```

Output

Here's the output:

```
***Template Method Pattern Demonstration.***
Computer Science course structure:
1. Mathematics
2. Soft skills
3. Object-Oriented Programming
Electronics course structure:
1. Mathematics
2. Soft skills
3. Digital Logic and Circuit Theory
```

Q&A Session

15.1 In this pattern, subclasses can simply redefine the methods based on their needs. Is this correct?

Yes.

15.2 In the abstract class `BasicEngineering`, only one method is abstract and the other two methods are concrete. What is the reason behind this?

I want the subclasses to override only the `courseOnSpecialPaper()` method here. Other methods are common to both courses, and they do not need to be overridden by the subclasses.

15.3 Suppose you want to add more methods in the `BasicEngineering` class, but you want to work on those methods if and only if your derived classes need them. Otherwise, you ignore them. This type of situation is common in some Ph.D. programs where some courses are mandatory, but if a student has certain qualifications, the student

may not need to attend the lectures for those subjects. Can you design this kind of situation with the Template Method pattern?

Yes, you can. Basically, you want to use a hook, which is a method that can help you to control some kind of flow in an algorithm.

What is a hook in programming? In very simple words, a hook helps you execute some code before or after existing code. It can help you extend the behavior of a program at runtime. A hook method can provide some default behavior that a subclass can override if necessary. Often, they do nothing by default.

To demonstrate this, let's add a few lines of code in the `BasicEngineering` class. Notice the bold lines in the following code segments:

```
// The previous code skipped
// The template method
public final void displayCourseStructure() {
 /*
 * The course needs to be completed
 * in the following sequence:
 * 1.Mathematics
 * 2.Soft skills
 * 3.Subclass-specific paper
 * 4.Additional paper, if any
 */
 // Common Papers:
 courseOnMathematics();//Step-1
 courseOnSoftSkills(); //Step-2
 // Course-specific Paper:
 courseOnSpecialPaper();//Step-3
 if (isAdditionalPaperNeeded()){
    courseOnAdditionalPaper();//Step-4 (if required)
 }
// The remaining code skipped
Where the hook method is defined as:
/* A hook method.
* By default, an additional
* subject is needed.
*/
boolean isAdditionalPaperNeeded(){
 return true;
}
```

These two code segments tell us that when you invoke the template method, by default the method `courseOnAdditionalPaper()` will be executed. It is because `isAdditionalPaperNeeded()` returns `true`, which in turn makes the `if`

condition inside the template method `true`. But in the upcoming demonstration, the `Electronics` class overrides this method as follows:

```
@Override
boolean isAdditionalPaperNeeded() {
  return false;
}
```

So, when you instantiate an `Electronics` instance and call the template method, you can see that `courseOnAdditionalPaper()` is NOT called at the end. Let's go through the complete program and output now.

Demonstration 2

Here's the modified implementation. The classes in this demonstration are in package `jdp3e.template_method.hook_example`. It helps you to get everything immediately when you download the source code from the Apress website.

```
// BasicEngineering.java
abstract class BasicEngineering {
     // The "Template Method"
     // Making the method final to prevent overriding.
     public final void displayCourseStructure() {
          /*
           * The course needs to be completed
           * in the following sequence:
           * 1.Mathematics
           * 2.Soft skills
           * 3.Subclass-specific paper
           * 4.Additional paper,if any
           */
          // Common Papers:
          courseOnMathematics();//Step-1
          courseOnSoftSkills(); //Step-2



          // Course-specific Paper:
          courseOnSpecialPaper();//Step-3
          if (isAdditionalPaperNeeded()){
               // Step-4 (if required)
                courseOnAdditionalPaper();
          }
     }
     private void courseOnMathematics() {
          System.out.println("1. Mathematics");
     }
```

```
     private void courseOnSoftSkills() {
          System.out.println("2. Soft skills");
     }
     /*
      * The following method will vary.
      * It will be overridden by the
      * derived classes.
      */
     public abstract void courseOnSpecialPaper();
     // Include an additional subject
     // if required.
     private void courseOnAdditionalPaper()
     {
          System.out.println("4. Compiler construction.");
     }
    /* A hook method.
     * By default, an additional



     * subject is needed.
     */
     boolean isAdditionalPaperNeeded(){
           return true;
     }
}
// ComputerScience.java
// The concrete derived class: ComputerScience
class ComputerScience extends BasicEngineering {
     @Override
     public void courseOnSpecialPaper() {
          System.out.println("3. Object-Oriented Programming");
     }
}
// Electronics.java
// The concrete derived class: Electronics
class Electronics extends BasicEngineering {
      @Override
      public void courseOnSpecialPaper() {
            System.out.println("3.Digital Logic and Circuit Theory");
      }
      /*
       * Overriding the hook method.
       * The additional paper is not needed
       * for the Electronics students.
       */
      @Override
      boolean isAdditionalPaperNeeded() {
```

```
            return false;
      }
}
// Client.java
class Client {
public static void main(String[] args) {
 System.out.println("***Template Method Pattern with a hook
                    method.***\n");
  BasicEngineering preferrredCourse = new ComputerScience();
  System.out.println("Computer Science course structure:");
  preferrredCourse.displayCourseStructure();
  System.out.println();
  preferrredCourse = new Electronics();
  System.out.println("Electronics course structure:");
  preferrredCourse.displayCourseStructure();
 }
}
```

Output

Here's the modified output:

```
***Template Method Pattern with a hook method.***
Computer Science course structure:
1. Mathematics


2. Soft skills
3. Object-Oriented Programming
4. Compiler construction.
Electronics course structure:
1. Mathematics
2. Soft skills
3. Digital Logic and Circuit Theory
```

Note You may prefer an alternative approach. For example, you could directly include the default method called `courseOnAdditionalPaper()` in `BasicEngineering`. After that, you could override the method in the `Electronics` class and make the method body empty. But is it a good idea to use an empty method in this example? I do not think so. For example, consider a case when you have many subclasses and most of them override the superclass method and you are forced to make this

method empty in each of them. You may say, instead of making a method body empty, you will throw an exception and indicate that it is not a valid method for a particular subclass. If you think like this, I suggest you recollect the LSP principle and its importance. This is why using a hook method can provide you with a better alternative in certain scenarios.

15.4 It looks like this pattern is similar to the Builder pattern. Is this correct?

No. Don't forget the core intent; the Template Method pattern is a behavioral design pattern, whereas the Builder pattern is a creational design pattern. In the Builder pattern, the client/customer is the boss. They control the order of the algorithm. In the Template Method pattern, you (or the developers) are the boss. You put your code in a central location (the abstract class `BasicEngineering.cs` in this example), and you have absolute control over the flow of the execution, which cannot be altered by the client. For example, you can see that "Mathematics" and "Soft skills" always appear at the top following the execution order in the template method `displayCourseStructure()`. The clients need to obey this flow.

If you alter the flow in your template method, other participants will also follow the new flow.

15.5 What are the key advantages of using a Template Method design pattern?

Here are some key advantages:

- You can control the flow of the algorithms. Clients cannot change this.
- Common operations are in a centralized location. For example, in an abstract class, the subclasses can redefine only the varying parts so that you can avoid redundant code.

15.6 What are the key challenges associated with a Template Method design pattern?

The disadvantages can be summarized as follows:

- The client code cannot direct the sequence of steps. If you want that type of functionality, use the Builder pattern (refer to Chapter **6**).
- A subclass can override a method defined in the parent class (in other words, hiding the original definition in the parent class), which can go against the Liskov Substitution Principle.
- Having more subclasses means more scattered code and difficult maintenance.

15.7 What will happen if a subclass tries to override the other parent methods in the `BasicEngineering` class?

This pattern suggests not to do that. When you use this pattern, you should not override all the parent methods entirely to bring a radical change in the subclasses. In this way, it differs from simple polymorphism.

15.8 It will be helpful if you explain some built-in template methods in Java.

Consider the `removeAll()` method of `java.util.AbstractSet`. It has the following definition:

```
public boolean removeAll(Collection<?> c) {
        Objects.requireNonNull(c);
        boolean modified = false;
        if (size() > c.size()) {
            for (Object e : c)
                modified |= remove(e);
        } else {
            for (Iterator<?> i = iterator(); i.hasNext(); ) {
                if (c.contains(i.next())) {
                    i.remove();
                    modified = true;
                }
            }
        }
        return modified;
    }
```

Have you noticed the use of the `size()` method ? On further investigation, you'll see that `size()` is present in the parent class `AbstractCollection` and it has the following definition:

```
public abstract int size();
```

Apart from this, there are many non-abstract methods in the `java.util.AbstractMap` and `java.util.AbstractSet` classes, which can also be considered as examples of the Template Method pattern. For example, notice the definition of the following method in the `AbstractMap<K,V>` class:

```
  public int size() {
        return entrySet().size();
    }
```

You can see that the previous method uses another method named `entrySet()`, which is defined as an abstract method in `AbstractMap<K,V>` as follows:

```
public abstract Set<Entry<K,V>> entrySet();
```

Previous chapter
< 14. Bridge Pattern

Next chapter
16. Observer Pattern >